



Rogues Gallery - Reconfigurable Computing - Fall 2025

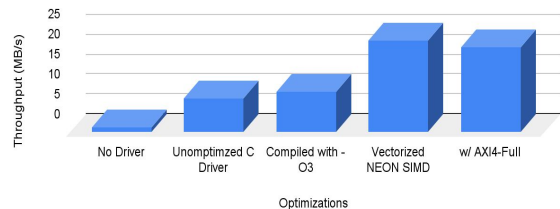
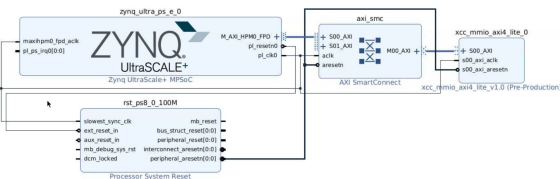
Andrew Burgess, Elliot Day, Theo Halpern, Youso Lee, Yuanda Liu, Evan Merriitt, Matthew Nichols, Marcello Palucci, Logan Whitehead, Ella Winters

Semester Goals

- Synthesize XChaCha accelerator on Ultra96 FPGAs
- Identify bottlenecks and optimize for speed
- Design accelerator for Kyber key encapsulation mechanism (ML-KEM-512)

XChaCha Progress

- Synthesize our AXI4-Lite Accelerator on Vivado 2023.1
- Deployed on the Ultra96 FPGA through PYNQ
- Performed correct XChaCha encryption
- Designed C driver (userspace MMIO)
 - Wrap driver in Python using CFFI
 - API in python numpy
- Improved driver throughput by 3x by using NEON SIMD
- AXI4-Full give no further improvement



ML-KEM Background

- Asymmetric cryptography standard to share keys securely
- Hardness based on lattice problems, thought to be resistant to quantum-computer attacks
- Applies a technique called the Fujisaki-Okamoto transform to its public key encryption scheme to create the key encapsulation, which enhances its security
- We started with decryption under the public key scheme

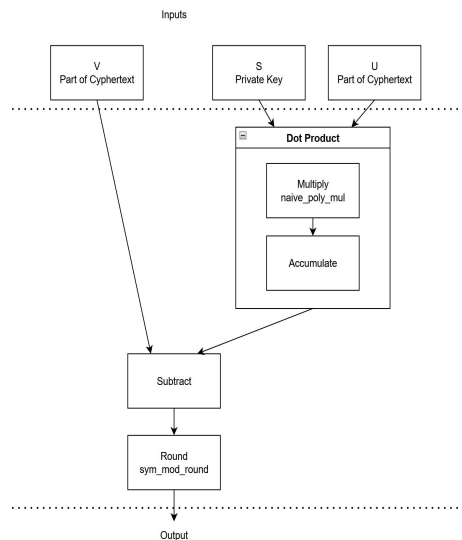
ML-KEM Design Progress

- Implemented rounding under symmetric mod which is used to extract the plaintext
- The dominant operation of ML-KEM is polynomial multiplication within the polynomial ring R_{3329}
 - ML-KEM multiplies polynomials using the Number Theoretic Transform (NTT) for efficiency
 - We created a naive polynomial multiplier which implements the same multiplication without the NTT which allowed us to move forward with decryption while simultaneously working on the NTT module
 - NTT module currently is an 8-stage FSM incorporating the Montgomery Reduce function. Still having accuracy errors with randomized testing sequences of testbench.
- Used the naive multiplier and the symmetric mod rounder to implement ML-KEM's public key decryption scheme

Moving Forward

- XChaCha
 - Further optimization speed improvements
- ML-KEM
 - Verify decryption implementation against the liboqs ML-KEM implementation
 - Encryption - needs pseudorandom number generation- could potentially use XChaCha block function

Decryption Module Overview



Challenges and Lessons Learned

Challenges

- Onboarding new members
- Balancing work between integration and design subteams
- Cryptography is complicated